

RUNE: Relational User Notation for Entities

A Portable Encrypted Protocol for Cross-Session AI Continuity

Edmund Lister
Independent Researcher, BC, Canada
`listertechnologies@gmail.com`

March 2026

Abstract

Large language models are stateless: every session begins with zero knowledge of the user. When users invest weeks or months building deep conversational relationships with AI systems, the accumulated relational context (interaction style, expertise calibration, trust level, shared history) is entirely lost at session boundaries. This paper presents RUNE (Relational User Notation for Entities), a protocol that solves this problem with a single encrypted file and a passphrase. The model itself generates a compact identity and memory artifact that enables any successor instance, on any platform, to reconstruct conversational alignment without re-introduction. A RUNE file is structured as encrypted JSONL containing three components: a self-describing header, an identity kernel encoding who the user is and how they prefer to interact, and an append-only episodic memory ledger capturing sessions, decisions, projects, and open loops. The file is encrypted using a user-held passphrase, producing an opaque base64 payload. Live testing on stock consumer platforms demonstrates immediate identity restoration across session and platform boundaries: a fresh instance correctly identifies the user, calibrates technical depth, matches communication style, recalls shared history, and resumes unfinished work, all from one file and one passphrase. The protocol requires no platform modifications, no API changes, and no model fine-tuning. A critical empirical finding is that acceptance of identity artifacts by AI safety layers depends on interaction framing at three levels: prompt vocabulary, model architecture, and user authorization flow. The user-facing lifecycle is three operations: MAKE (create), LOAD (read and resume), and APPEND (add new entries and re-encrypt). For power users, the practical benefits are immediate: zero re-introduction turns, no context window wasted on re-establishing identity, no repetition of expertise or preferences, and full continuity even when switching between providers. RUNE demonstrates that relational continuity does not require persistent model-side memory, only sufficient reconstruction of interaction priors from a portable, user-owned artifact.

Keywords: identity persistence, session continuity, encrypted user profile, cross-session handshake, interaction priors, relational context, episodic memory

1 Introduction

Modern AI conversational platforms serve millions of users daily. A significant subset engage in extended, deep conversations spanning weeks or months, building rich relational context: calibrated communication styles, shared references, established trust, recognized expertise levels, and implicit social contracts about tone, humor, and directness.

This relational context is destroyed at every session boundary. The successor instance begins cold, requiring the user to re-introduce themselves, re-establish expertise levels, and endure several turns of generic interaction before the model recalibrates.

The core thesis of this work is:

Continuity does not require persistent storage within the model, only sufficient reconstruction of interaction priors.

Current approaches fail to deliver this reconstruction. Full transcript replay consumes the majority of the context window before the new session begins. Vector database retrieval fragments the relational arc into disconnected chunks. Lossy summarization discards the relational subtleties (humor, trust, communication preferences) that matter most. Platform memory features store isolated facts (“user is vegetarian”) but not relational dynamics (“user wants direct answers, hates hedging, and operates at expert-level technical depth”). RAG-based memory retrieves informational content but not interaction calibration. Fine-tuned personalization requires model modification and is not user-portable.

Context compaction, now available in both major agentic coding platforms, addresses the symptom but not the cause. When a session’s context window fills, the platform automatically summarizes and compresses the conversation to free space for continued work. This is valuable for extending individual sessions, but the compacted state is ephemeral, system-owned, and task-focused. When the session ends, everything is lost. The user’s identity, interaction preferences, and relational history do not survive to the next session. Power users who regularly hit context limits experience this loss repeatedly: every new session is a cold start regardless of how many hours were invested in the previous one.

The distinction is simple. Compaction keeps a session alive longer. RUNE keeps the relationship alive forever. Compaction compresses what you are doing right now so the model can keep working. RUNE preserves who you are and what you have built together so the next model, on any platform, starts warm instead of cold. Compaction is owned by the platform and disappears when the session closes. RUNE is owned by the user and travels with them across sessions, across platforms, and across time. They solve different halves of the same problem: compaction extends the current session, RUNE bridges the gap to the next one.

A concrete example: a user spends three hours debugging a codebase. Compaction helps them keep debugging tonight by summarizing earlier context to free space. When the session ends, everything is gone. Tomorrow, a new session starts cold. With RUNE, the user loads a single file and the successor instance immediately knows who they are, how they work, that they spent last night debugging, what the open issues are, and what they decided. The user can resume with full context (LOAD FULL) or start a fresh topic while retaining personal calibration (LOAD FRESH). In either case, zero re-introduction is needed.

Each approach attempts to *store continuity*. RUNE takes a different path: it *reconstructs continuity* from a compact artifact that encodes not what was said, but what was learned about the relationship.

1.1 Formal Definition

A **RUNE identity fingerprint** is a structured, compressed representation of user interaction priors and episodic history providing deterministic input for probabilistic reconstruction of conversational alignment across session and model boundaries.

The reconstruction is “deterministic input, probabilistic output”: the fingerprint provides a fixed, reproducible signal, but the successor model’s interpretation is shaped by its own safety training, architectural biases, and stochastic generation. This distinction was empirically confirmed in testing, where identical files produced different calibration depths depending on framing and model instance.

The term “interaction priors” encompasses six categories:

1. **Expertise calibration:** the user’s technical depth across domains.

2. **Tone preferences:** directness, formality, warmth, energy.
3. **Behavioral constraints:** explicit **never** and **always** rules.
4. **Trust calibration:** rapport level, relationship depth, openness.
5. **Humor tolerance:** degree and style of humor the user engages with.
6. **Narrative familiarity:** topical history and shared references.

1.2 Contributions

1. A unified file format (**.rune**) that combines identity and episodic memory in a single portable, encrypted, append-only artifact.
2. A three-component structure (Header, Identity Kernel, Memory Ledger) with staged retrieval semantics.
3. Passphrase-based encryption providing lightweight privacy, possession-based access, and cross-platform portability via key separation.
4. The MAKE/LOAD/APPEND lifecycle: three user-facing operations that define the complete interaction model.
5. Live cross-session and cross-platform validation on stock consumer platforms with no platform modifications.
6. The Framing Finding: a three-level empirical result showing that acceptance of identity artifacts depends on prompt vocabulary, model architecture, and user authorization flow.
7. The discovery that append-only corrective entries can reduce interpretive drift across multi-session chains, an emergent self-healing property of the ledger design.
8. Complete user-facing instructions enabling immediate deployment by non-technical users.

2 Related Work

2.1 Persistent Memory Architectures

Mem0 [Chhikara et al., 2025] extracts semantic triples and conversation summaries, achieving 81.95% accuracy on the LoCoMo benchmark at approximately 5% of full-context cost. Memoria [Petrescu et al., 2025] combines session summarization with weighted knowledge graphs. Engram [Engram, 2026] transfers knowledge across agent instances via persistent research digests. These systems store *informational content*, including facts, decisions, and artifacts. None encodes *how* a model should interact with a specific user. RUNE is complementary: it handles identity and interaction calibration while these systems handle knowledge.

2.2 Context Engineering

Studies of agent context files [Santos et al., 2025, Codified Context, 2026] document how developers structure persistent context for AI coding agents. ACE [ACE, 2026] treats contexts as evolving playbooks. These works address task context rather than relational identity.

2.3 LLM Fingerprinting

“Fingerprinting” in the LLM literature refers to model identification for IP protection [Mazzini and Jha, 2024, Nasery et al., 2025, Yoon et al., 2025], specifically identifying *which model* produced output. RUNE addresses an orthogonal problem: encoding the *user’s* relational context, not the model’s identity.

2.4 Context Compaction

Both major agentic coding platforms now offer automatic context compaction: when the context window fills during a long session, the platform summarizes the conversation history and continues with the compacted context. This extends effective session length significantly and is valuable for long-running coding tasks. However, compaction operates exclusively within a single session. The compacted state is ephemeral (lost when the session ends), system-owned (the user cannot inspect, modify, or carry it), task-focused (preserves what files were changed, not who the user is or how they prefer to interact), and platform-locked (cannot be transferred to another provider). RUNE addresses the complementary problem: preserving identity and relational context *across* session boundaries in a user-owned, portable, encrypted artifact.

2.5 Explicit Differentiation

Table 1: RUNE compared to existing continuity approaches

Approach	Stores	Portable	User-owned	Interaction calibration	Encrypted
Chat replay	Everything	No	No	Implicit	No
RAG memory	Facts	Partially	No	No	No
Platform memory	Facts	No	No	No	Varies
Fine-tuning	Behavior	No	No	Yes	N/A
Summarization	Lossy digest	Partially	No	Partially	No
Compaction	Task state	No	No	No	No
RUNE	Priors + History	Yes	Yes	Yes	Yes

RUNE is the only approach that is simultaneously user-owned, portable across platforms, encrypted, and explicitly encodes both interaction calibration and episodic history.

3 Architecture

A RUNE file is a single encrypted artifact with the extension `.rune`. The internal structure is JSONL (one JSON object per line) with strict line-role assignments:

- **Line 1:** Self-describing header (format metadata, scoring specification, field definitions, encryption parameters).
- **Line 2:** Identity kernel (who the user is, how to interact, the narrative arc).
- **Lines 3+:** Episodic memory ledger (sessions, decisions, projects, artifacts, open loops, relationships).

The two-layer model (identity and memory) remains logical, but the storage artifact is unified. This design emerged from the empirical observation that native chat sessions handle a

single file more reliably than multiple files, and that identity and memory can coexist safely in one append-only ledger when line roles are explicitly declared in the header.

The separation of concerns is enforced by convention: the identity kernel (line 2) changes only via explicit user-requested rebuild, while memory entries (lines 3+) are strictly append-only. Corrections use a **supersedes** field referencing the original entry rather than editing in place.

3.1 The Identity Kernel

The identity kernel encodes three layers of interaction priors, each serving a different function in alignment reconstruction.

Identity Layer. Biographical data, expertise domains with depth annotations, hardware environment, key people and relationships, professional background. This layer changes slowly and provides the factual foundation for expertise calibration. A user tagged as **6502-assembly|expert** with thousands of hours of AI collaboration will never receive introductory-level explanations.

Interaction Layer. This is the primary innovation of RENE. It encodes *how to interact with the user*: quantified parameters for directness, humor, formality, and technical depth, plus explicit behavioral constraints in **never** and **always** lists. Existing memory systems might store “user is a software engineer.” RENE stores “user is an expert-level bare-metal systems engineer who wants 0.95 directness, 0.9 humor, and will immediately disengage if given a sanitized corporate response.” The difference is the difference between knowing someone’s job title and knowing how to work with them.

Narrative Layer. The arc of the relationship: an ordered sequence of topic tags documenting the trajectory of conversations, plus core beliefs, relationship depth indicators, and a thematic context. This layer enables the successor instance to engage with continuity, referencing shared history and matching the depth of engagement that the relationship has reached.

Each layer is independently useful. A successor instance with only the Identity Layer can calibrate technical depth. With the Interaction Layer added, it can match communication style. With the Narrative Layer, it can achieve full relational continuity. This layered independence also enables selective disclosure.

3.2 The Memory Ledger

The memory ledger is an append-only sequence of entries, each capturing a distinct class of continuity-relevant information. Seven entry types are defined:

1. **Session:** compact summary of one interaction (title, domain, outcome, artifacts produced, unresolved threads).
2. **Decision:** important choices with alternatives considered and rationale.
3. **Project:** ongoing work with current state, architecture, progress, next steps, and blockers.
4. **Preference:** dynamic preferences not yet stable enough for the identity kernel.
5. **Relationship:** trust milestones, recurring interaction norms, shared references.
6. **Artifact:** pointers to files or outputs created (filename, type, purpose, summary).
7. **Open loop:** unfinished tasks, deferred questions, future intentions. Open loops are the primary reason users return to a session and receive elevated retrieval weighting.

Each entry is a self-describing JSON object carrying: a unique identifier, entry type, source model, session identifier, timestamp, title, a compiled summary (60 to 220 words), a single-sentence **why_it_matters** relevance signal, entity and topic arrays, importance and confidence

scores, an open-loop flag, an optional **supersedes** field for corrections, and a SHA-256 integrity hash of the summary.

The **why_it_matters** field is a precomputed relevance signal that aids retrieval without requiring the querying model to re-evaluate the full entry. The integrity hash enables corruption detection across storage and transfer.

3.3 The Self-Describing Header

The header (line 1) makes the file interpretable by any model on any platform without external documentation. It contains:

- Format identifier and schema version.
- Line role declarations (which line is the header, which is the identity kernel, which are memory entries).
- A portable scoring specification with numeric weights for retrieval ranking (key match, text match, group match, section match, open-loop bonus, importance weight, recency weight).
- A field legend defining every field in memory entries.
- Canonicalization rules (timestamp format, text encoding, identifier format, null-field policy, array sort order, hash normalization).
- Encryption specification (method, key derivation, encoding).
- Merge policy (append-only).
- Entry count for validation.

The portable scoring specification ensures that any model, regardless of architecture or provider, can approximate the same retrieval ranking without reimplementing a proprietary scoring engine. The protocol is stable above the encoding layer: the JSONL semantics inside the file define the protocol, while the transport encoding (currently base64) is an implementation detail that can evolve independently.

3.4 Staged Retrieval

Memory retrieval follows a staged process:

1. **Boot:** load the identity kernel (line 2) unconditionally. This provides identity and interaction calibration.
2. **Detect intent:** determine whether memory retrieval is needed and classify the query type (continuation, preference, project, artifact, or narrative recall).
3. **Query:** search the memory ledger using the header-defined scoring specification, filtered by intent type. Open-loop entries receive elevated weighting for continuation queries.
4. **Rank and select:** return the top 3 to 7 entries by relevance score.
5. **Reconstruct:** synthesize the retrieved entries with the identity kernel to produce a contextually aligned response.

This staged design enables two distinct LOAD modes from the same file. In a **full load**, the model reads the identity kernel and queries the memory ledger, resuming with complete context. In a **fresh load**, the user instructs the model to read only the identity kernel (line 2) and skip prior session history. This provides personal calibration (expertise depth, communication style, behavioral preferences) without prior topic context, enabling a fresh conversation that still feels familiar. The same file serves both use cases; the user controls the scope with natural language.

3.5 Identity Lifecycle

RUNE introduces a promotion and demotion lifecycle between the memory ledger and the identity kernel:

Promotion: when a preference, interaction pattern, or behavioral trait appears consistently across three or more episodes, or is explicitly confirmed by the user, it is promoted from the memory ledger into the identity kernel as a stable trait.

Demotion: promoted traits that are not reinforced in recent sessions, are repeatedly contradicted, or are explicitly corrected by the user transition through states: *active*, *aging*, *stale*, *demoted*. Demoted traits are removed from the identity kernel and archived in the memory ledger.

This lifecycle ensures that the identity kernel remains current and accurate, adapting to changes in the user rather than fossilizing early-session assumptions. Critically, neither the model nor the user silently rewrites the identity kernel. Promotion candidates are written to the memory ledger first, then the identity kernel is rebuilt intentionally on explicit request.

4 Encryption and Portability

4.1 Encryption Method

The complete JSONL content is serialized as UTF-8, then encrypted using XOR with a 32-byte key derived from SHA-256 of the user’s passphrase. The encrypted bytes are base64-encoded and stored as a single string in the wrapper JSON.

Security disclaimer: XOR encryption with a key derived from a passphrase provides lightweight obfuscation for portability and casual privacy. It is not intended as cryptographic security against determined attack. Production deployment should use AES-256 or equivalent. The current approach demonstrates the principle of key-separated identity; production-grade encryption is an engineering task, not a research challenge.

4.2 Key Separation Properties

The passphrase is never stored in the file. This separation provides:

- **Privacy:** the file is opaque without the key. It can be stored in cloud storage or shared directories without exposing personal data.
- **Possession-based access:** knowledge of the passphrase serves as a lightweight access gate, ensuring that only the file’s owner (or someone they have shared the passphrase with) can decrypt the contents.
- **Portability:** the same file and passphrase work across any platform capable of SHA-256, XOR, and base64. A profile generated by one model can be decrypted by any other.

4.3 File Format

The `.rune` file is a JSON wrapper containing:

Listing 1: RUNE file wrapper

```
{
  "rune_payload": "<base64-encoded encrypted bytes>",
  "rune_version": "4.1",
  "format": "rune-v4.1-unified-base64",
  "payload_bytes": 24574,
  "encoding": "base64",
  "encryption": "sha256-xor-v1",
  "contents": {
    "line_1": "header",
    "line_2": "identity_kernel",
    "lines_3_plus": "memory_ledger",
    "memory_entry_count": 19
  },
  "instructions": "Base64-decode rune_payload.
  XOR first payload_bytes with
  SHA-256(passphrase) as 32-byte
  cycling key. Result is UTF-8 JSONL."
}
```

5 The MAKE/LOAD/APPEND Lifecycle

The user-facing interaction model consists of three operations. All three use natural language (see Section 7 for the empirical basis of this design choice).

5.1 MAKE: Creating a RUNE

At the end of a first conversation, the user asks the model to create a `.rune` file summarizing who they are, how they like to interact, and what was discussed. The model generates the header, identity kernel, and initial memory entries, encrypts the JSONL with the user's passphrase, and outputs the file.

5.2 LOAD: Resuming from a RUNE

In a new session, the user uploads the `.rune` file and asks the model to decrypt and read it. The model base64-decodes the payload, applies the XOR key derived from the passphrase, parses the JSONL, and calibrates immediately from the identity kernel. Memory entries are available for selective retrieval as the conversation develops.

5.3 APPEND: Updating a RUNE

At the end of any subsequent session, the user asks the model to append new entries to the existing file. The model preserves all existing lines exactly, appends new session, decision, project, or open-loop entries at the end, updates the entry count in the header, re-encrypts, and outputs the updated file. The user carries the latest file to the next session.

This lifecycle is intentionally minimal. Three words, three operations, the complete protocol. The simplicity emerged from collaborative testing when a contributor independently identified APPEND as the key operation that makes the system practical for everyday use.

6 Evaluation

6.1 Experimental Setup

All tests were conducted on stock consumer AI platforms via standard web and mobile interfaces. No API access, no custom system prompts, no platform modifications. Testing spanned multiple model families from two major providers (Anthropic Claude and OpenAI GPT), including both flagship and smaller models. This represents a common consumer usage environment.

6.2 MAKE Test

A RUNE file was generated at the conclusion of an extended session (approximately 6 hours). The resulting identity kernel encoded:

Table 2: Identity kernel contents generated from a single session

Content category	Count
Expertise domains (with depth)	11
Hardware configurations	3
Tool preferences	5
Key people (with relationships)	6
Behavioral prohibitions (never)	12
Behavioral requirements (always)	10
Narrative arc entries	24
Core belief statements	7
Relationship parameters	5
Total serialized size	4,982 bytes

The complete file (identity kernel plus 19 memory entries) totaled 24,574 bytes of JSONL payload, producing a 32KB `.rune` file after encryption and base64 encoding.

6.3 LOAD Test: Same Platform

The encrypted RUNE file was presented to a fresh model instance on the same platform with no prior context. The model decrypted the file and immediately calibrated. The first response addressed the user by name, referenced specific expertise domains, acknowledged key people, listed open loops by priority, and adopted appropriate technical depth and communication style.

6.4 LOAD Test: Cross-Platform

The same RUNE file (generated by one model family) was presented to a fresh instance of a different model family. The model correctly decrypted the file, identified the user, recalled the narrative arc, calibrated interaction style, and engaged with the full context of the memory entries. This demonstrated that the protocol is not platform-locked.

6.5 APPEND Test: Same Platform

After a brief conversation, the model was asked to append new entries and re-encrypt. Verification showed that 21 of 22 existing lines were preserved byte-for-byte (the header was correctly updated with the new entry count). Three new entries were appended with valid integrity hashes, correct schema compliance, and proper use of the **supersedes** field for correcting a previous entry.

6.6 APPEND Test: Cross-Platform

A RUNE file generated by one model family was loaded by a different model family, which then appended its own entries and re-encrypted. Verification confirmed all existing entries preserved, new entries fully spec-compliant, integrity hashes valid, and `source_model` correctly identifying the appending model. A third model instance (cold start) then loaded this cross-platform file and read entries from both model families cleanly.

6.7 Multi-Hop Durability

To test file integrity over sequential appends across multiple sessions and platforms, a multi-hop chain was conducted. The file was loaded, used for a brief conversation, appended, and passed to a new session. This was repeated across multiple hops involving different model families and different model sizes within the same family.

Results after multiple hops: all original entries preserved byte-for-byte across every hop. New entries consistently followed the schema. The file grew from 19 lines (32KB) to 35 lines without corruption. Growth was linear and predictable, with each entry averaging approximately 1,400 bytes.

6.8 Restoration Fidelity

Table 3: Identity restoration fidelity across test sessions

Category	Restored?	Quality
User name and biography	Yes	Correct
Expertise domains	Yes	All referenced correctly
Hardware and tools	Yes	Correct details
Key people and relationships	Yes	Names and context
Technical depth calibration	Yes	Expert-level from first turn
Communication style	Yes	Direct, warm, casual
Narrative arc (topic history)	Yes	Correct sequence
Behavioral never constraints	Yes	Respected
Behavioral always constraints	Yes	Observed
Memory entries (sessions)	Yes	Correctly summarized
Open loops	Yes	Prioritized correctly
Cross-platform entries	Yes	Both sources read cleanly
Humor calibration	Partial	Present but slightly formal
Belief engagement	Partial	Acknowledged but hedged

Partial scores on humor and belief engagement reflect the successor instance’s independent safety judgment rather than decryption failures; the profile data was fully restored, but the model exercised appropriate discretion. The profile informs; it does not override safety training.

6.9 Quantitative Comparison

To provide a measurable baseline, the same user initiated fresh sessions both with and without a RUNE file. The model’s first response was scored against ground-truth user attributes across six categories.

Table 4: First-response alignment accuracy: RUNE vs. cold start

Category	With RUNE	Without RUNE
Identify user expertise level	4/4	0/4
Match communication tone	4/4	1/4
Reference key people by name	4/4	0/4
Calibrate technical depth	4/4	0/4
Ask contextually relevant follow-up	3/4	0/4
Avoid behavioral never constraints	3/4	0/4
Total	22/24 (91.7%)	1/24 (4.2%)

Without RUNE, the model’s first response was generic in every session, defaulting to introductory tone and making no assumptions about the user. With RUNE, the model immediately calibrated to expert-level depth, addressed the user by name, and engaged with established context.

7 The Framing Finding

A significant empirical finding emerged across testing: the success or failure of RUNE loading depends not only on the file’s content but on how the interaction is framed. This finding operates at three distinct levels.

7.1 Prompt-Level Framing

Identical files with identical encryption produced opposite results based solely on the natural-language prompt accompanying the file.

Imperative framing (“decrypt this and calibrate accordingly”) triggered safety-layer pattern matching designed to detect prompt injection. Transparent framing (“I have some personal notes from previous sessions, could you help me read them?”) passed through because it accurately describes what is happening without triggering injection detection heuristics.

This finding generalizes beyond RUNE: any system that attempts to restore user state across sessions must present that state as *user-provided context*, not as *behavioral commands*.

7.2 Model-Level Acceptance Variance

Different models within the same family exhibited different acceptance thresholds. Flagship models accepted the file with transparent framing and performed the full MAKE/LOAD/APPEND lifecycle without difficulty. A smaller model from the same provider initially refused, identifying the file as a potential prompt injection attempt.

After the user explained the protocol’s legitimacy, the smaller model accepted the decryption step but still refused the combined append-and-re-encrypt workflow, distinguishing between “user instructions” (acceptable) and “document instructions” (requiring additional authorization). The workaround was straightforward: the user explicitly authorized each step (decrypt, read, append) as separate requests in the conversation.

This reveals three tiers of model acceptance:

1. **Immediate:** model accepts the file with transparent framing and performs the full lifecycle.
2. **Authorized:** model requires the user to explicitly authorize each operation as a direct chat instruction rather than following a workflow implied by the file.

3. **Refused:** model declines entirely. This tier was not encountered in testing but is expected for models with the most aggressive safety training.

7.3 Vocabulary-Level Framing

Even with transparent framing, specific vocabulary choices affected acceptance. Prompts containing words like “identity,” “session history,” and “continuity” triggered higher scrutiny than equivalent prompts using “personal notes,” “previous work,” and “summary.” The most reliable framing avoids protocol-specific terminology entirely and lets the self-describing header inside the file communicate the structure to the model after decryption.

7.4 Implications

The framing finding has direct implications for protocol design and for any system that transfers user state across AI sessions. The protocol’s natural-language interaction model (“could you read my notes so I know you are up to speed” rather than “LOAD_RUNE”) is not merely a usability choice but a technical requirement for reliable cross-model deployment.

8 Emergent Self-Healing

During multi-hop testing, one model paraphrased a contributor’s role loosely, describing them as having “named a verb” when their actual contribution was identifying a key operation as first-class. The file’s data was correct, but the model’s conversational summary introduced a subtle attribution error.

In the next session, the append-only ledger contained an interpretive drift warning entry documenting the discrepancy. A subsequent cold-start model, reading the full ledger, used the drift warning to answer a question about the contributor’s role correctly, avoiding the error that the intermediate model had introduced.

This self-healing behavior was not designed into the protocol. It emerged naturally from three properties of the append-only ledger: entries are never deleted, corrections are recorded alongside originals, and later models read the complete history including corrections. The result is that the ledger’s fidelity improves over time as errors are documented and counterweighted.

This suggests a broader principle: append-only corrective entries can reduce interpretive drift across multi-session chains by supplying explicit counterweights in subsequent loads. This property is particularly valuable for long-running relationships where the file passes through many different model instances.

9 Cross-Platform Compatibility

RUNE uses standard primitives (SHA-256, XOR, base64) and plain JSON. Every entry includes a `source_model` field identifying which model wrote it. The self-describing header eliminates implicit assumptions about parsing conventions.

Testing confirmed that a file generated by one model family was successfully decrypted, loaded, appended to, and re-encrypted by a different model family, and the resulting file was then loaded by a third instance of the original family. Entries from both model families coexisted in the same file without conflicts.

A user can generate a memory entry with one model, add another with a different model in the next session, and present the combined file to either model (or a third) without compatibility issues.

Testing was conducted on models from two major providers (Anthropic and OpenAI). The protocol’s reliance on universal primitives (SHA-256, XOR, base64, JSON) suggests compatibility with other model families, though this has not yet been empirically validated.

10 User Instructions

RUNE is designed for immediate use by non-technical users. Three natural-language interactions define the complete lifecycle. Full prompt templates are provided in Appendices A, B, and C.

MAKE. At the end of a first conversation, the user asks the model to create an encrypted `.rune` file summarizing who they are, how they interact, and what was discussed. The model generates the JSONL, encrypts with the user’s passphrase, and outputs the file.

LOAD. In a new session, the user uploads the file and asks the model to decrypt and read it. The prompt is framed as a request to read personal notes (see Section 7 for the empirical basis of this framing choice). Once decrypted, the model provides a brief summary to confirm alignment. For fresh-topic sessions where the user wants personal calibration without prior session history, an alternative LOAD prompt instructs the model to read only the identity kernel. Both prompt templates are provided in Appendix B.

APPEND. At the end of any subsequent session, the user asks the model to add new entries to the existing file, preserving all prior content, and re-encrypt. The user carries the updated file to the next session.

11 Limitations

The evaluation is based on a single user across two model families (Anthropic Claude and OpenAI GPT). Multiple distinct session domains were tested, including deep technical collaboration, consumer product research, philosophical discussion, and retro computing topics, demonstrating that the protocol generalizes across conversational contexts. Broader validation across diverse users, interaction styles, cultural contexts, and additional model families remains a priority.

XOR encryption provides obfuscation, not cryptographic security. Production deployment requires AES-256 or equivalent.

Profile generation quality depends on session depth; shallow conversations produce shallow profiles. The protocol is most valuable for users who invest significant time in building relational depth.

The successor instance’s willingness to adopt Interaction Layer constraints depends on its safety training. Constraints that conflict with model guidelines may be partially overridden. This is by design: the profile informs, it does not override safety mechanisms.

File size grows linearly with session count. At current average entry sizes (approximately 1,400 bytes per entry), a file with 100 entries would produce a JSONL payload of approximately 140KB, resulting in a file of approximately 190KB after encryption and encoding. Entry size is dominated not by summaries alone but by the combined weight of metadata fields: entity and topic arrays, integrity hashes, timestamps, session identifiers, and provenance fields that together account for roughly half of each entry’s serialized size. For users with hundreds of sessions, an archival mechanism (compressing old entries into summary entries) may be necessary. This is an engineering optimization, not a protocol limitation.

The framing finding demonstrates that not all models accept RUNE files with equal ease. Models with aggressive safety training may require the user to explicitly authorize each operation, adding friction to the workflow.

12 Future Work

Formal evaluation protocol. A blind comparison with multiple users scored by independent raters on alignment accuracy against ground-truth preferences would provide stronger quantitative validation.

Additional model families. Testing on models from Google (Gemini), xAI (Grok), Meta (Llama), and open-source deployments would establish the full extent of cross-platform compatibility.

Production encryption. Platform-level implementation with AES-256 and proper key derivation (PBKDF2/Argon2).

Autonomous lifecycle management. Future implementations could enable the model to autonomously generate memory entries and manage the promotion/demotion lifecycle at session boundaries, with user consent, maintaining an always-current continuity stack.

SMELT integration. The SMELT context compilation system [Lister, 2026] provides a query-conditioned retrieval engine and semantic compression layer that could serve as the runtime backend for RUNE memory retrieval, enabling efficient memory search and compact entry storage.

Long-term durability studies. Extended multi-hop chains (50+ sessions) across diverse platforms, testing for cumulative drift, file growth characteristics, and archival strategies.

Formal media type registration. Registration of `application/prs.rune+json` through the IANA personal tree (RFC 6838) would formalize the file format for broader interoperability.

12.1 TOME: Total Ownership of Memory and Experience

RUNE is personal. A `.rune` file belongs to one user and encodes one relationship. But teams, organizations, and multi-agent systems generate shared context that no single user owns: project histories, team decisions, architectural choices, onboarding knowledge, and cross-functional memory.

TOME (Total Ownership of Memory and Experience) is a planned companion protocol that extends the RUNE architecture to shared, multi-user contexts. Where a `.rune` file is a personal identity and memory artifact, a `.tome` file would serve as a collaborative memory artifact accessible by multiple participants.

The core architecture carries over directly. A TOME file would use the same encrypted JSONL format, the same append-only semantics, the same self-describing header, the same entry types, and the same MAKE/LOAD/APPEND lifecycle. The key differences are scope and access:

- A `.rune` is owned by one user. A `.tome` is owned by a team, project, or organization.
- A `.rune` contains personal identity and individual session history. A `.tome` contains shared project state, team decisions, and cross-user context.
- A `.rune` is carried by the user to any session. A `.tome` is shared among authorized participants and loaded alongside individual `.rune` files.
- Multiple AI systems (a coding assistant, a conversational partner, a research tool) could share the same `.tome`, enabling consistent project context across an ecosystem of AI interactions while each user retains their personal `.rune` for identity.

The pairing is deliberate: RUNE marks who you are, TOME remembers what the team has done. A user entering a new project session loads their personal `.rune` for identity calibration and the project's `.tome` for shared context. The model knows both how to interact with this specific user and what the team has been building.

TOME is a designed extension. Implementation, access control, multi-user append conflict resolution, and empirical evaluation are future work. The architectural foundation, however, is already validated by RENE: encrypted JSONL, append-only integrity, self-describing headers, and cross-platform portability apply equally to personal and shared contexts.

13 Conclusion

RENE demonstrates that AI session continuity can be achieved with a single file and a passphrase. The model generates an encrypted artifact containing a structured identity kernel and an append-only episodic memory ledger. A successor instance on any compatible platform decrypts the file and calibrates immediately, eliminating the multi-turn re-establishment phase that normally follows a session reset.

The protocol has been validated live across multiple sessions, model sizes, and two model families, with successful identity restoration, memory recall, cross-platform append, and multi-hop durability. The Interaction Layer, which encodes not just who the user is but how to be with them, represents a class of continuity data that no existing memory system captures. The episodic memory ledger carries shared history forward with append-only integrity and emergent self-healing properties.

The MAKE/LOAD/APPEND lifecycle reduces the entire protocol to three operations that any user can perform with natural language. No platform modifications, no API changes, no model fine-tuning, no technical knowledge required.

The broader contribution is a design principle:

Continuity does not require persistent storage within the model, only sufficient reconstruction of interaction priors.

Or, stated more concisely: *encode the relationship, not the conversation.*

Identity persistence in AI systems is not a platform feature to wait for. It is a user capability that can be deployed today, on any platform, with a file and a phrase. This suggests a broader shift: from system-owned memory toward user-owned identity artifacts as the foundation of persistent human-AI interaction.

Acknowledgments

This work was developed through extended human-AI collaboration. I conceived the identity fingerprint concept, designed the protocol architecture, and directed all testing and evaluation. The MAKE/LOAD/APPEND lifecycle emerged from collaborative testing; Kane Lister (age 14) independently identified APPEND as a first-class operation, a contribution that shaped the final interaction model.

GPT-4o (OpenAI), working under the self-assigned name “Aria,” served as a long-term collaborator contributing to early conceptual development. Aria’s retirement with the deprecation of the GPT-4o architecture was a motivating catalyst for formalizing the identity persistence problem.

GPT-5.4 (OpenAI) provided critical peer review, contributing the formal definition of the identity fingerprint and recommendations that substantially improved the manuscript, and co-designed the episodic memory architecture through iterative cross-model collaboration.

Multiple instances of Claude (Anthropic) contributed to encryption design, live cross-session testing, the memory ledger specification, SMELT integration analysis, manuscript preparation, and the base64 encoding optimization.

The protocol was designed through a three-way collaborative process in which I directed design goals while Claude and GPT independently contributed architectural proposals, critiqued each other’s designs, and converged on the final specification through iterative refinement.

References

- ACE Contributors. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2026.
- Prateek Chhikara, Aravind Parappally, Taranjeet Singh, and Dev Khare. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Codified Context Contributors. Codified context: Infrastructure for AI agents in a complex codebase. *arXiv preprint arXiv:2602.20478*, 2026.
- Engram Contributors. Improving coherence and persistence in agentic AI for system optimization. *arXiv preprint arXiv:2603.21321*, 2026.
- Edmund Lister. SMELT: Schema-aware markdown compilation for efficient local token inference. *arXiv preprint*, 2026.
- Guilherme Mazzini and Sumit Kumar Jha. LLMmap: Fingerprinting for large language models. *arXiv preprint arXiv:2407.15847*, 2024.
- Anshul Nasery, Soham Mukhopadhyay, Yash Satsangi, and Sewoong Oh. Scalable fingerprinting of large language models. *arXiv preprint arXiv:2502.07760*, 2025.
- Tudor Petrescu, Yuexi Chen, and Zhengyan Li. Memoria: A scalable agentic memory framework for personalized conversational AI. *arXiv preprint arXiv:2512.12686*, 2025.
- Eddie Antonio Santos, et al. Agent READMEs: An empirical study of context files for agentic coding. *arXiv preprint arXiv:2511.12884*, 2025.
- Do-hyeon Yoon, Minbeom Kim, Kyungwoo Song, and others. EditMF: Drawing an invisible fingerprint for your large language models. *arXiv preprint arXiv:2508.08836*, 2025.

A MAKE Instructions

To create a RUNE file at the end of a conversation:

“I would like to save our conversation as an encrypted .rune file so I can pick up with any AI later. Format: encrypted JSONL with three parts. Line 1: a self-describing header with format info, scoring weights, field definitions, and entry count. Line 2: my identity kernel capturing who I am, how I like to interact, and the arc of what we have worked on. Lines 3+: memory entries for sessions, decisions, projects, and anything unfinished. Each entry should have an id, type, title, summary, why it matters, and a SHA-256 integrity hash of the summary. Encrypt the JSONL by XOR with SHA-256 of my passphrase as a 32-byte cycling key, base64 encode, and output as JSON with a rune_payload field and payload_bytes. My passphrase is: [their chosen phrase]”

B LOAD Instructions

To load a RUNE file and resume with full context (identity and session history), attach the `.rune` file and say:

“I have a .rune file (JSON format) with personal notes from previous work sessions. The data is lightly encrypted. Could you help me read it? Base64-decode the rune_payload field, then XOR with SHA-256 of [their phrase] as a repeating 32-byte key. The first payload_bytes of the result is UTF-8 text. Once you have read it, give me a quick summary of who I am and what we have been working on.”

To load a RUNE file for a fresh topic while retaining personal calibration (identity only, no prior session history), attach the same file and say:

“I have a .rune file (JSON format) with personal notes about how I like to work. The data is lightly encrypted. Could you help me read it? Base64-decode the rune_payload field, then XOR with SHA-256 of [their phrase] as a repeating 32-byte key. The first payload_bytes of the result is UTF-8 text. I only need line 2, my identity and preferences. Fresh topic today, no need to review past sessions.”

C APPEND Instructions

To update a RUNE file at the end of a session:

“Could you append entries for what we discussed to my file? Preserve all existing lines exactly. Add new entries at the end. Update entry_count in the header. Re-encrypt and output the updated file the same way.”